The graph indicates that much of the CPU time is idle, implying that the server has adequate processing capacity to meet the present workload. The CPU time is divided into smaller amounts for both system-level and user-level activities, while the IO wait time is negligible. This distribution in equilibrium represents an efficient working of the system with no major CPU and IO bottlenecks.

Real-time monitoring of this kind is crucial for the early detection of performance anomalies, allowing for proactive scaling and resource optimisation in high-traffic situations.

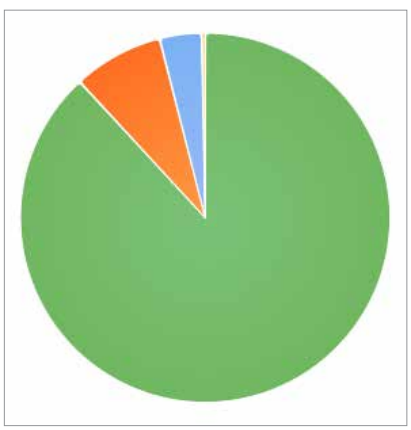

Figure 1: CPU usage metrics

Green: cpu_usage_metrics.Idle %
Blue: cpu_usage_metrics.User %
Red: cpu_usage_metrics.System %
Orange: cpu_usage_metrics.IO Wait %

CPU core-wise usage metrics: Figure 2 illustrates the CPU core-wise usage data over the past hour, captured and aggregated at 10-second intervals. The query retrieves the mean values of *idle*, *iowait*, *nice*, *steal*, *system*, and *user* states from the cpu_corewise_usage_ metrics measurement, grouped by each CPU core.

The pie chart displays the distribution of CPU time across various cores, where each segment represents either the *idle* or *user* time of a specific core. It can be seen that most cores are predominantly idle, while selected cores exhibit notable user-space activity. This visualisation provides an effective overview of the system's workload distribution and highlights any imbalances or underutilisation across the cores.

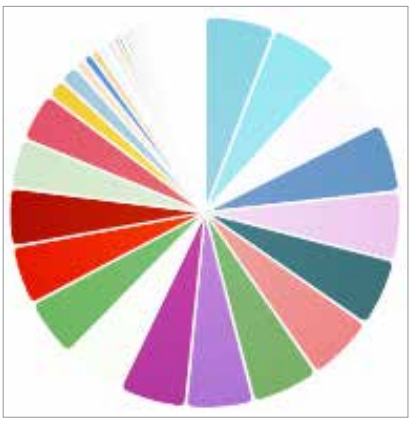

Figure 2: CPU core-wise data

Cyan: Idle – Core 9
Teal: Idle – Core 11
Violet: Idle – Core 3
Blue: Idle – Core 5
Pink: Idle – Core 7
Gray: Idle – Core 15
Brown: Idle – Core 17
Orange: Idle – Core 1
Dark Red: Idle – Core 13
Dark Green: Idle – Core 2
Black: Idle – Core 0
Light Red: Idle – Core 8
Purple: Idle – Core 10
Magenta: Idle – Core 6
Red: Idle – Core 4
Yellow-Brown: User – Core 4
Steel Blue: User – Core 6
Sky Blue: User – Core 0

API metrics: Response count over time: Figure 3 shows the response count of different API end-points by time. The data is taken from the api_metrics measurement by counting the number of response_time entries in 10-second intervals, organised by the corresponding endpoint. Zero values are imputed for gaps in the data to have a regular time series.

The line graph provides a representation of response volume for various API endpoints, giving information about traffic trends, periods of peak activity, and comparison of the endpoints. Tracking these trends is critical in performance assessment, load distribution, and timely detection of anomalies in distributed systems.

Aggregated logs: Message count over time: Figure 4 shows the time distribution of message counts, divided by message type. Every line on the plot represents one particular log message, with the y-axis reflecting the total occurrences per each 20-second time slice and the x-axis representing time. The chart identifies fluctuations in the occurrence of various log messages so that times of increased system activity, patterns that are new, and anomalies can be identified. Temporal visualisation of logs is important for tracking system health and diagnosing system problems.

Mean response time: The provided Grafana visualisation is as shown in Figure 5. The mean response time trends are given for multiple API endpoints over a defined time window. Each coloured line represents a distinct API endpoint, such as:

- _api_v1_auth_login,
- _api_v1_auth_your-participations,
- _api_v1_event_get-events, and
- _api_v1_event_get-hostedEvents.

The graph aggregates data at regular intervals of 10 seconds, enabling real-time observation of API performance. Noticeable spikes in the plot, especially for the login endpoint, indicate occasional latency surges. This visualisation helps in identifying potential bottlenecks and monitoring the consistency of API response times.

Memory data: The pie chart visualisation in Figure 6 illustrates the distribution of various memory usage metrics collected over a one-hour time window, averaged at 10-second intervals. The segments represent different categories